

Master MLDM - First year - December 15, 2025

Introduction to Artificial Intelligence - Exam on Prolog

Time allocated: 3h - The subject is 4 pages long - No documents allowed.

TAKE CARE: any cheating will be severely punished and will lead to a formal complaint to the disciplinary council of the university.

1 Proof tree (5 points)

Consider the Prolog program below:

- (1) $p1(A,B,C) :- p2(A,B), p3(B), p3(C).$
- (2) $p2(A,B) :- p4(A), p4(B).$
- (3) $p2(A,B) :- p4(B), p4(A).$
- (4) $p3(A) :- p5(A), p4(A).$
- (5) $p4(1).$ (6) $p4(2).$ (7) $p4(3).$ (8) $p4(4).$
- (9) $p5(2).$ (10) $p5(3).$

1. Draw the proof tree of the resolution of the goal: $?- p1(1,Y,Z).$ and give all the solutions for this goal.
2. Suppose we put a cut between $p4(A)$ et $p4(B)$ in the second clause of the program. Show, on the tree you built at the previous question, which branches are pruned during the resolution of the goal: $?- p1(1,Y,Z).$ and give again all the solutions for this goal.
3. Suppose we also put a cut between $p5(A)$ et $p4(A)$ in the fourth clause of the program. Show, on the tree you built at the previous question, which branches are pruned during the resolution of the goal: $?- p1(1,Y,Z).$ and give again all the solutions for this goal.

2 Writing functions in Prolog (2 points)

The dragon function is defined by:

$$dragon(x,y,z) = \begin{cases} 1 & \text{if } x = 0, y = 0 \text{ and } z = 0 \\ 2 & \text{if } x = 1, y = 0 \text{ and } z = 0 \\ 3 & \text{if } x = 0, y = 1 \text{ and } z = 0 \\ dragon(x - (y + z), y, z) + dragon(x, x - (y + z), z) & \text{if } x > y + z \text{ and } z \geq 0 \\ dragon(x, dragon((y + z) - x, y, z), dragon(x, (y + z) - x, z)) & \text{if } x < y + z \text{ and } z < 0 \\ dragon(x, dragon(1, 2, 3) \times dragon(4, 5, 6), dragon(8, 9, 10)) & \text{if } x > 0 \end{cases}$$

The knight function is defined by: $knight(a,b) = \begin{cases} a & \text{if } a = b \\ knight(a - b, knight(b, a)) & \text{if } a > b \\ knight(b, b) \times knight(a, a) & \text{if } b > a \end{cases}$

1. Write the Prolog program that defines a predicate dragon/4 where $dragon(A,B,C,Res)$ is true if Res is the result of the dragon function of A, B and C .
2. Write the Prolog program that defines a predicate knight/3 where $knight(M,N,Res)$ is true if Res is the result of the knight function of M and N .

3 Unification (2 points)

For each of the goals below, say whether it is true or false. In cases where it makes sense, give the value(s) of the variable(s) that make(s) a goal true.

1. $?- X \text{ is } Y+1, Y=4.$
2. $?- [p(X), date(september, 15, 2025), Y, [a, Z, b], _] = [p(d), date(M, 15, Y), T, [H|T1]|T2].$
3. $?- p(p(p(nil, 3), D), q(q(A, 3), 4)) = p(p(B, 2), q(q(q(nil, 2), E), C)).$
4. $?- g(A, f(3, H), k(B, H), s(H, A, T, A)) = g(3, f(B, 7), k(A, T), s(T, B, 7, H)).$
5. $?- X = 2+1.$
6. $?- [[[[[9|T], 3], 8], 2], G] = [[A|B], D].$
7. $?- f(Y, k(A, 3), g(3, [1, X, 3, Z]), X) = f(2, Z, g(C, [A, X, C, Z]), h(5, 1)).$
8. $?- [E|[A, B], D]] = [[[[[9|_], 3], 8], 2], C].$

4 Knowledge base modeling and querying (3 points)

Below is information about a simple world, for any person P1, P2 and P3.

Convert this information into a Prolog program:

- If a person P1 is richer than another person P2, and they live in the same place then P1 pays more taxes than P2.
- If a person P1 is richer than another person P2, and P1 lives in town and P2 lives in the countryside, then P1 is smarter than P2.
- If a person P1 is richer than a person P3 that is richer than a person P2, then P1 is richer than P2 (the relationship "richer than" is transitive).
- The relationship "younger than" is also transitive.
- If a person P1 is smarter than a person P2, or if P1 is younger than P2 and P1 is a student and P2 is a teacher, then P1 is happier than P2.
- If P1 pays more taxes than P2 then P1 is jealous of P2.
- If P1 is happier than P2 or P1 is jealous of P2, then P1 has feelings.
- Bess and Dana live in the same place.
- John is a student.
- John is younger than Mary.
- Suzy is a teacher.
- Cody lives in town.
- Cody is richer than Dana.
- Dana lives in the countryside.
- Bess is richer than Cody.
- Mary is younger than Suzy.

Considering the Prolog program you just wrote, convert the following english queries into Prolog goals (do not try to solve those goals, this is not the question):

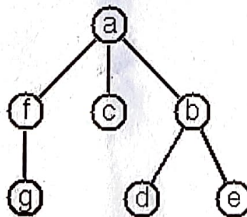
1. Who is richer than whom?
2. Who has feelings?
3. Who is jealous of whom?
4. Who is smarter than whom?

5 Trees in Prolog (4 points)

In this exercise we consider n-ary trees where a tree is represented by a compound term `t(Value,List)` where Value is the root value of the tree `t` and List is the list of all the subtrees of `t`. For example, the compound term:

`t(a,[t(f,[t(g,[])])],t(c,[]),t(b,[t(d,[]),t(e,[])])])`

is a Prolog representation of the following tree:



Write the Prolog program specifying the following predicates:

1. `is_a_tree/1`. `is_a_tree(T)` is true if `T` is a tree of the form `t(V,L)` where `V` is a value and `L` is a list of trees, possibly empty. For example:
`?- is_a_tree(t(a,[t(f,[t(g,[])])],t(c,[]),t(b,[t(d,[]),t(e,[])])]).`
`true.`
`?- is_a_tree(t(a,[t(f,[t(g,h)])],t(c,[]),t(b,[t(d,[]),t(e,[])])]).`
`false.`

2. `count_nodes/2`. `count_nodes(T,N)` is true if `T` is a tree that has `N` nodes. For example:
`?- count_nodes(t(a,[t(f,[t(g,[ ]])],t(c,[ ]),t(b,[t(d,[ ]),t(e,[ ]])])),N).`
`N = 7`
`?- count_nodes(t(a,[t(f,[t(g,[ ]])],t(c,[ ]),t(b,[t(d,[ ]),t(e,[ ]])])),7).`
`true.`
`?- count_nodes(t(a,[t(f,[t(g,[ ]])],t(c,[ ]),t(b,[t(d,[ ]),t(e,[ ]])])),8).`
`false.`
3. `length_internal_path/2`. `length_internal_path(T,N)` is true if in the tree `T` the sum of the lengths of the paths to each of the nodes is equal to `N`. For example:
`?- length_internal_path(t(a,[t(f,[t(g,[ ]])],t(c,[ ]),t(b,[t(d,[ ]),t(e,[ ]])])),L).`
`L = 9`
4. `depth_first/2`. `depth_first(T,L)` is true if `L` is the list of the nodes of the tree `T` traversed depth first. For example:
`?- depth_first(t(a,[t(f,[t(g,[ ]])],t(c,[ ]),t(b,[t(d,[ ]),t(e,[ ]])])),L).`
`L = [g, f, c, d, e, b, a].`

6 DCG (4 points)

Consider the grammar of a small programming language $\mathcal{P}$

```

program => statements
statements => statement ';'
statements => statement ';' statements
statement => variable ':=' exp
statement => 'if' condition 'then' 'begin' statements 'end' 'else' 'begin' statements 'end'
statement => 'while' condition 'do' 'begin' statements 'end'
statement => 'read' '(' variable ')'
statement => 'write' '(' exp ')'
condition => exp operatorComp exp
exp => exp1 '+' exp
exp => exp1 '-' exp
exp => exp1
exp1 => exp2 '*' exp1
exp1 => exp2 '/' exp1
exp1 => exp2
exp2 => '(' exp ')'
exp2 => '0'
exp2 => '1'
exp2 => '2'
exp2 => '3'
exp2 => variable
variable => an atom
operatorComp => '<'
operatorComp => '>'

```

for example, it can be used to generate/parse a program in the $\mathcal{P}$ language such as:

```

x42b := 3*(y+1) ;
while (x42b < 1) do begin
  if (x > y)
  then
    begin
      x42b := x42b+1 ;
    end
  else
    begin
      x42b := 2 ;
    end ;
end ;

```

1. Write a DCG that can be used to say whether a program in the $\mathcal{P}$ language is syntactically correct with respect to the above grammar. Note that there exists a built-in predicate called `atom/1` such as `atom(X)` is true if `X` is an atom.
2. Provide two different ways to write the goal you have to run to say whether the program `x42b := 3*(y+1) ;` is syntactically correct or not.
3. Modify your DCG to build a tree representation of any syntactically correct program of the $\mathcal{P}$ language. For example, the tree representation of:

```

if (x > y)
then
  begin
    x42b := x42b+1 ;
  end
else
  begin
    x42b := 2 ;
  end ;

```

should be the compound term:

```

statementList(
  ifThenElse(
    comparison(>,variable(x),variable(y)),
    statementList(
      affect(
        variable(x42b),
        plus(variable(x42b),digit(1))),
      null),
    statementList(
      affect(variable(x42b),digit(2)),
      null)),
  null)

```

4. More generally, give the Prolog clause generated from the following DCG rule after loading it into the Prolog workspace:
`p(A) --> q(C),[a,b],r,{s(A,C)},[c],t(-,C).`